

Midterm Assignment: Prompt Engineering

Prompt Engineering Course

Ankara Medipol University

Submitted by: Mehmet Eren Tutaş

Student Number : 64230037

Department : Computer Engineering

Submission Date: 15.11.2025

Part 1: Prompt Comparison Analysis

AI tool(s) used: Gemini

a. Domain & Task Description

Domain: Engineering

Task Description: Compare the key differences in code execution performance and compilation speed between Java and Python, analyzing why these differences exist for a software engineering application.

b) Version A: Vague Prompt

Java vs Python speed comparison

Java is generally **significantly faster** than Python for most CPU-intensive and performance-critical applications. This difference is rooted in how the languages are executed:

Execution Mechanism

- Java is Compiled and uses a JIT Compiler:** Java code is compiled into **bytecode** and then executed on the **Java Virtual Machine (JVM)**. The JVM uses a **Just-In-Time (JIT)** compiler to dynamically translate the bytecode into highly optimized, native machine code during execution. This dynamic optimization allows Java to achieve very high speeds, especially in tasks that run for a long time (like enterprise applications).
- Python is Interpreted:** Standard Python (CPython) is an interpreted language. The code is executed line-by-line by the interpreter, which adds runtime overhead. Python also uses **dynamic typing** (type checks happen at runtime), which is more flexible for developers but adds to the execution time compared to Java's **static typing** (type checks happen at compile time).

Performance In Use Cases

The speed comparison often dictates the typical use cases for each language:

- Java is preferred for high-performance, large-scale applications:**
 - Enterprise-level systems:** Due to its robust performance, scalability, and stability.
 - Android backends and development** (though Kotlin is also popular).
 - High-throughput APIs** and applications requiring strong multithreading capabilities.
- Python is preferred where development speed and specific ecosystems are key:**
 - Rapid prototyping and development:** Its concise syntax and high-level nature mean less code and faster time-to-market.
 - Data Science, Machine Learning, and AI:** While Python itself is slower, its core numerical libraries (like NumPy and Pandas) are implemented in highly optimized C/C++, making data processing tasks extremely fast.
 - Scripting, automation, and basic web APIs** where I/O (Input/Output) speed, not raw CPU speed, is the bottleneck, often making the language difference less relevant.

In short, Java has better **execution speed**, while Python offers better **developer speed** (faster development time).

c) Version B: Improved Prompt

Act as a Senior Software Engineer and performance analyst. Your task is to provide a detailed technical comparison between Java and Python, focusing only o...

Düşünme sürecini göster

JSON

```
{
  "comparison_summary": {
    "java_execution": "Generally faster, especially for long-running, CPU-bound applications.",
    "python_execution": "Generally slower for CPU-bound tasks due to its interpreted nature.",
    "java_compilation": "Explicit Ahead-of-Time (AOT) compilation from '.java' source files.",
    "python_compilation": "Implicit compilation from '.py' source to '.pyc' bytecode at runtime."
  },
  "detailed_analysis": [
    {
      "factor": "Compilation Model",
      "java_explanation": "Java uses a sophisticated Just-In-Time (JIT) compiler within the JVM, which optimizes code during execution.",
      "python_explanation": "The standard CPython implementation is primarily an interpreter, executing code line-by-line."
    },
    {
      "factor": "Type System",
      "java_explanation": "Java is **statically typed**. All variable and object types are checked at compile time.",
      "python_explanation": "Python is **dynamically typed**. The type of a variable is determined at runtime."
    },
    {
      "factor": "Concurrency & Parallelism",
      "java_explanation": "Java has robust, OS-level multi-threading built in. The JVM manages threads efficiently.",
      "python_explanation": "CPython uses a **Global Interpreter Lock (GIL)**, which restricts true parallelism in CPU-bound tasks."
    }
  ],
  "conclusion": "Prioritize **Java** when raw execution speed is a critical requirement for CPU-intensive tasks."
}
```

d. Comparison Analysis

The difference in output quality between Version A and Version B is dramatic, highlighting the impact of advanced prompting techniques . Version A ("Java vs Python speed comparison") produced a generic, high-level paragraph, likely similar to a standard blog post. It lacked technical depth, actionable detail, and structure, making it unsuitable for a computer engineering student. Its usefulness was minimal.

In contrast, Version B produced a highly specific, useful, and structured analysis . The improvements are directly attributable to the prompt's design .

Three concrete improvements are:

1. **Structured Output:** The use of a JSON format forced the AI to organize information logically, separating the high-level summary from the detailed factor-by-factor analysis. This makes the output immediately useful for comparison.
2. **Technical Specificity:** By using **Role Prompting** ("Act as a Senior Software Engineer") and **Audience Specification** ("second-year computer engineering students"), the AI adjusted its technical vocabulary. It correctly focused on concepts like JIT compilation, static vs. dynamic typing, and the CPython GIL, rather than simplistic analogies.
3. **Constraint-Following:** The **Constraint** ("focusing *only* on code execution performance and compilation speed") prevented the AI from discussing irrelevant metrics like development speed or library availability, which the vague prompt likely included.

Ultimately, Version B worked better because it explicitly defined the *context*, *task*, *audience*, *constraints*, and *output format*. It transformed the AI from a general conversationalist into a specialized analyst, demonstrating that effective prompt engineering is about precise instruction, not just asking a question .

Part 2: Practical Prompt Design

a. Framework Selection & Justification

Framework Selection: CO-STAR Framework Business Task: Create a social media content calendar for one month

Justification: The CO-STAR framework is ideal for this creative and strategic task . A successful content calendar requires more than just instructions; it needs a clear understanding of the brand's identity and goals. CO-STAR provides the perfect structure: 'Context' defines the business, 'Objective' sets the monthly goal, 'Style' and 'Tone' establish the brand voice, and 'Audience' targets the content precisely . 'Response' then defines the exact deliverable (the calendar), ensuring the AI's output is perfectly aligned with the business's marketing strategy.

b. Complete Prompt Following Framework Structure

[CONTEXT] : We are a small software startup that has just launched a new mobile game called "Pixel Rush". The game is an endless runner with a retro pixel-art style. We have limited marketing budget and need to grow organically.

[OBJECTIVE] : The primary goal for this one-month content calendar is to drive new user acquisition. Specifically, we want to increase downloads of "Pixel Rush" by 30% and grow our Instagram/Twitter follower count.

[STYLE] : Fun, energetic, and engaging. The visual style should be vibrant, heavily featuring pixel art and screenshots from the game. The copy should be witty and clever.

[TONE] : Approachable, familiar, and humorous. We are part of the gaming community, not a formal corporation. Use Gen-Z references, humor, and memes where appropriate, but remain professional (no profanity).

[AUDIENCE] : Young professionals (ages 22-30) who are active on social media (primarily Instagram and Twitter). They are casual gamers who play mobile games during their commutes or work breaks. They appreciate nostalgia (pixel art) and good humor.

[RESPONSE] : Generate a 4-week (one month) social media content calendar in a Markdown table format.

c) AI OUTPUT

• Week 1: The Hook (Awareness) • Week 2: The Community (Engagement) • Week 3: The Hype (Desire) • Week 4: The Push (Action)					
🚀 "Pixel Rush" 4-Week Content Calendar					
Week	Day	Theme	Post Concept (Instagram)	Post Concept (Twitter)	CTA / Goal
1	Mon	Launch Day!	Reel: High-energy, 15-second montage of the best gameplay moments (near misses, big scores, cool power-ups). Text: IT'S HERE. Pixel Rush is LIVE. Your commute just got 110% less boring.	GIF: Fast-cut GIF of gameplay. Copy: "Pixel Rush" is officially OUT. ⚡ We're not saying you should play it during your meeting, but we're not <i>not</i> saying that. Download link in bio!	DOWNLOADS (Link in Bio)
1	Wed	Relatable Humor	Carousel (Meme): 1. "My boss thinks I'm taking notes." 2. (Screenshot of "Pixel Rush" gameplay) 3. "My notes:" (Screenshot of "GAME OVER" screen).	Meme: Use the "Distracted Boyfriend" or similar format. Boyfriend: Me Girlfriend: My actual work Other Woman: "Pixel Rush"	Followers / Shares (Tag a coworker)
1	Fri	Core Mechanic	Reel/Carousel: "How to not be a noob at Pixel Rush"	Thread (3 tweets): 1. "New to Pixel Rush?"	Engagement / Retention

d. Refinement & Iteration

The initial output from the CO-STAR prompt was a comprehensive 4-week calendar. However, it exhibited two common weaknesses : the example captions were generic, and the content mix was heavily weighted toward "Gameplay," lacking sufficient community engagement. The captions failed to capture the specific "Gen-Z" tone we requested, and the "Community" pillar, essential for organic growth, was underutilized.

The refined follow-up prompt addressed these issues directly. It explicitly instructed the AI to rewrite captions to emphasize the *nostalgic pixel-art style* and to replace four "Gameplay" posts with "Community" focused ideas, providing specific examples like high-score challenges and contests .

The resulting output was significantly stronger. The captions were wittier and more brand-aligned. The calendar now had a clear strategy for User-Generated Content (UGC) and interaction, directly supporting the objective of building a community, not just broadcasting advertisements. This iterative process demonstrated the importance of refining an initial AI output, moving from a generic "good" response to a specific, "excellent," and strategically-aligned asset.

🎮 "Pixel Rush" 4-Week Content Calendar (Refined)					
Week	Day	Theme	Post Concept (Instagram)	Post Concept (Twitter)	CTA / Goal
1	Mon	Launch Day!	Reel: High-energy 10-second montage of the fastest, most chaotic gameplay (near-misses, big combos). Text: IT'S ALIVE. 🎮 We bottled 16-bit nostalgia and pure, unadulterated panic. Your thumbs will thank us (or hate us). "Pixel Rush" is LIVE.	GIF: A perfect loop of the main character's run cycle. Copy: We pixel-perfected every jump and polished every 8-bit sound just to ruin your productivity. Sorry, not sorry. "Pixel Rush" is OUT. Download link in bio!	DOWNLOADS (Link in Bio)
1	Wed	Relatable Humor	Carousel (Meme): 1. "My to-do list." 2. (Long, boring text). 3. "My brain:" 4. (Intense, fast-paced 'Pixel Rush' screenshot). Copy: It's called <i>priority management</i> . This pixelated world isn't gonna save itself.	Text-Only Meme: "My brain during a 3 PM Zoom call: 🎮 blip-blop-blip-blip 🎮 (it's the 'Pixel Rush' theme song, I am one twitchy thumb away from faking a bad connection)"	Followers / Shares (Tag a coworker)
1	Fri	Nostalgia (NEW)	Static Image: A screenshot of 'Pixel Rush' next to a '?' block. Copy: "This game was inspired by the 90s. The fast parts. The hard parts. The 'throw your controller' parts.	Image + Question: "Honest question: What 90s game does this screenshot give you vibes of? We're getting 'Sonic meets... something we're probably not allowed to say.'	Community / Engagement (Comment your answer)

e. Framework Effectiveness Reflection

The CO-STAR framework was highly effective for structuring this prompt . For a creative task like a content calendar, defining the 'Style', 'Tone', and 'Audience' was crucial. Without them, the AI would have produced generic, corporate content. The 'Objective' (increasing downloads) anchored the entire calendar to a measurable business goal.

I would not use a different framework for this task . A framework like RACI would be inappropriate as it focuses on roles, not creative output. CO-STAR excels at briefing an AI for marketing or content creation. This process taught me that framework selection is about matching the tool to the task. CO-STAR is the right tool when the *identity* and *voice* of the output are just as important as the *information* it contains.

Part 3: Critical Reflection

a. Skill Progression

Initially, my understanding of prompt engineering was rudimentary . I believed it was simply about writing short, concise sentences to get a factual answer, similar to a basic search query. My perspective has evolved significantly; I now understand it as a systematic process of instruction, constraint, and context design to guide an AI toward a specific, high-quality output. My primary 'aha moment' was realizing the immediate power of role prompting. Seeing how drastically the AI's response shifted in tone, depth, and technical accuracy in Part 1—simply by assigning it a "Senior Software Engineer" persona—was a breakthrough. It clarified that I was not just *asking* a question but *scripting* a response from a specific expert. The most challenging concept was Chain-of-Thought (CoT). I initially struggled to see its value, but after designing the structured Version B prompt , I overcame this. I realized that forcing the AI into a logical JSON structure was an implicit CoT, compelling it to break down its reasoning step-by-step, which produced a far more robust analysis.

b. Practical Application

A primary real-world scenario in my software engineering career will be code generation and debugging. Instead of writing boilerplate code from scratch, I can use prompt engineering to generate entire modules or functions. For example, I could task the AI to "Act as a senior Java developer. Write a thread-safe singleton class for database connection

pooling." For debugging complex algorithms, I can provide the faulty code block and use a **Chain-of-Thought** prompt, asking the AI to "Analyze this code line-by-line and identify the logical error causing the off-by-one result." The techniques I would use are **Role Prompting** (for specialized code) and **Few-Shot Examples** (providing a sample of my existing codebase to match the style). The main challenge will be the AI's potential to 'hallucinate' non-existent libraries or generate code that is syntactically correct but logically flawed or insecure. I would address this by always treating AI-generated code as a 'first draft' that requires rigorous human review and testing.

c. Limitations & Ethics

When prompting, I must consider two critical limitations of LLMs . First is the **lack of true context or 'common sense'**. An AI might generate code that is technically functional but nonsensical in a real-world business application, or it might introduce a subtle but critical security flaw because it doesn't understand the *implications* of its code. Second is the **knowledge cut-off and data bias** . The AI cannot know about a zero-day vulnerability or a new library released after its training date, potentially providing outdated or insecure code. The primary ethical consideration in my code generation scenario is **accountability**. If I use AI-generated code and it fails, causing a data breach or system crash, who is responsible? The ethical line is clear: the engineer is always accountable. To ensure responsible use , I must treat the AI as a 'co-pilot', not an oracle. All AI-generated code must be rigorously tested, validated, and understood by me before it is ever committed to a production environment.

d. Goals for the Second Half

In the second half of the course, I want to explore advanced prompt efficiency —how to write shorter, faster, and more effective prompts that yield maximum results with minimal input. My primary skill gap is managing AI unpredictability. I aim to master techniques for handling AI hallucinations and errors, ensuring my prompts are not just creative but also reliable and robust.